

Summary of Replit → Render Prep

Created on June 22, 2026 by [Summary AI](#)

Action Items

Festus Confirm production secrets and provide API inventory to Replit

Ask Replit to list all configured secrets/APIs and confirm each one is set to production (Stripe, email API, other third-party keys). Provide Replit with the definitive list of APIs you are using and confirm which secrets are still in test/sandbox mode and need rotation before launch.

Festus Request HIPAA/confidentiality review and role enforcement from Replit

Ask Replit to verify database residency and confidentiality controls, confirm whether Replit can support HIPAA-compliant hosting for the database or advise how to segregate sensitive data; request checks to ensure role separation (students cannot access teacher data) and that no unprotected URLs expose private information.

Festus Implement email provider (Resend) and 2FA setup

Create a Resend account, obtain and store the Resend API key in production secrets, and configure email-driven 2FA for users (email-only 2FA initially). Test the flow in sandbox using test keys, then move to production keys when validated.

Speaker C Create tasks and run integration tests (Stripe, roles, unregistered access)

Document tasks in the task board: verify Stripe API integration (test then live), confirm correct role assignments and separation for student/teacher/admin, test that unregistered users cannot access private endpoints, and run automated/manual tests across these scenarios; report back with results and blockers.

Speaker C Prepare GitHub → Render migration plan

Set up or verify the GitHub repository and CI workflow that will push changes to Render; outline steps to migrate the database to Neon/Postgres and to update deployment/process so changes pushed from Replit → GitHub are reliably shipped to Render for production.

Weston Send COPPA / school onboarding guidance

Share the COPPA-style guidance and checklist referenced in the meeting to the team (what to show schools, what student data to collect, parental consent considerations) so the product and legal teams can implement compliant onboarding and data limits for under-13 accounts.

Overview

- Team reviewed Replit workspace and checklist for going live: **secrets, API integrations, database residency, 2FA, email delivery**.
- Primary recommendation: keep development/prototyping on Replit but **migrate production workloads (DB + deploy) to Render/Neon + GitHub** when ready; Render provides better stability and HIPAA-capable hosting and is inexpensive in practice (**Render bill noted ~\$118 some months; Neon free for light usage**).
- Immediate risks: **secrets set to test vs production**, possible **unprotected URLs** exposing data, and **role/permission gaps** that may allow students to view teacher data.

- Compliance & onboarding constraints: COPPA/US rules for under-13 students require limited data collection and/or school/parent consent; 2FA/email flows must be implemented carefully to avoid blocking onboarding.

Replit & secrets

- Replit currently hosts the development workspace and its built-in database; the UI shows which secrets exist but may not reveal the database URL.
- Team must confirm each secret (Stripe, email providers, other APIs) is set to **production keys** (not sandbox/test) before going live.
- Replit's AI tooling can scan the project, surface APIs in use and highlight vulnerabilities (example: one project had **7 high vulnerabilities** reported); use that to produce an inventory of APIs and secrets.
- Recommended next step: ask Replit to produce a full list of configured APIs/secrets and identify any secrets still in test mode; then mark which must be migrated or rotated for production.
- Outlook: once secrets and API inventory are verified, the team can finalize which services remain on Replit (prototyping) and which move to production hosting.

Database & hosting

- Current state: database hosted in Replit (cloud). Team discussed moving DB to Neon/Postgres and hosting app on Render for improved reliability and HIPAA options.
- Typical workflow proposed: Replit pushes → GitHub (revision control) → GitHub → Render (production deployment). This preserves revision history while moving runtime to a robust host.
- Cost & stability: Render + Neon can be far cheaper and more stable than relying on Replit for production; Render has logs, billing visibility and easier troubleshooting (one participant paid **\$118** and had unused balance after two months; Render free tiers exist; next paid tier noted **\$25/month**).
- Consideration: moving DB away from Replit prevents bottlenecks and makes it easier to enable HIPAA controls and monitoring.
- Outlook: keep Replit for rapid iteration; prepare migration plan to GitHub + Render/Neon for production once security/compliance checks pass.

Payments (Stripe)

- Current state: project currently only had AI integration; Stripe was not implemented yet for the reviewed account.
- Requirements: implement Stripe as the single payment platform for paid plans; make sure Stripe keys in secrets are **production/live** (not sandbox) and confirm billing plan/configuration for Pro / Campus / Enterprise tiers.
- Suggested approach: integrate using test keys first (sandbox), validate payment flows (use test card 4242...), verify plan assignment and then switch to live keys and confirm payment plan status and billing on Stripe.
- Enterprise flow: enterprise customers should not be presented with direct payment prompts; they will contact sales for quotes and onboarding.
- Outlook: once Stripe is integrated and validated in test mode, flip to live keys before public launch and confirm payment logs and reconciliation.

Security, roles & 2FA

- Key issues identified: role separation (student vs teacher vs admin) must be enforced so students cannot access teacher scoring or sensitive data; unprotected URLs could leak data to unauthenticated users.
- Replit can run security checks and surface vulnerabilities (security settings → security view). Team saw pointers to where vulnerabilities appeared and which secrets exist.
- 2FA: team recommended requiring **2FA at login for all accounts** (not just admins) to protect student data and prevent spam signups; but watch COPPA/onboarding friction for under-13 users (some accounts may need school-mediated onboarding rather than self-serve 2FA enforcement).
- Data protection: for any patient-like / sensitive student data, the database and access controls must be HIPAA-compliant if applicable; roles must never cross-access data.
- Outlook: run a focused security review (secrets, URL access control, role enforcement) and fix any unprotected endpoints before migration to production.

Email & notifications (Resend)

- Current state: no email management was configured; team recommended using an email API (Resend) to send 2FA codes and transactional emails.
- Resend parameters: free tier provides **3,000 transactional emails/month**; marketing/other uses may be additional (~**1,000** extra for marketing in example). Get and store Resend API key as a production secret.
- Implementation: wire Resend for 2FA codes (email-only 2FA option suggested) and for onboarding/confirmation flows; test using the sandbox/test flow then use production keys.
- Outlook: once Resend account/API key is set, confirm delivery, rate limits and spam mitigation; forward or centralize emails if team wants them routed to a Gmail or shared inbox.

COPPA / minors & onboarding

- Rules summary: under-13 users require restrictions—schools may onboard students (school authorization) without parent consent if the school provides COPPA-style notice; self-serve child signup should be avoided unless parental consent or limited data collection is in place.
- Requirements: provide a school COPPA-style notice describing what student data is collected and how it is used; limit personal data stored for under-13 accounts and avoid collecting last names or other PII unless necessary.
- Outlook: keep under-13 accounts restricted until the team decides the onboarding model (school-led vs parent consent) and implements the minimal allowed data collection in the DB.

Testing, QA & deployment workflow

- Testing: Replit and its AI tooling can generate sample users and run automated tests; suggestion included creating **100 sample students** and **500 sample teachers** with varied grades to stress-test role separations and UI flows.
- QA responsibility: one participant (AJ referenced) will do manual testing while Replit runs automated checks; Replit can create mockups, UI suggestions and run test scenarios programmatically.
- Deployment: recommended path is to keep rapid iteration on Replit, push code to GitHub for revision control, then deploy to Render (production). Render provides logs and support for quick fixes.
- Outlook: run full integration tests (payments, emails, role access, unregistered URL access) before switching to live keys and routing production traffic.

Timeline & next steps (outlook)

- Short-term: verify secrets (live vs test), confirm Stripe + Resend configuration in test mode, validate role separation and 2FA flows, and run vulnerability/security checks in Replit.
- Medium-term: migrate DB and production deploys to Render/Neon via GitHub once the MVP is stable and pilot schools are lined up; monitor costs and log/incident workflows on Render.
- Follow-up: schedule a checkpoint after task creation and initial fixes (team proposed re-evaluating progress next week).